

W25 CMPUT 412/503 Exercise 2

ROS Development and Kinematics

TA's: Adam, Monta, Dikshant, and Jasper
Written report due: Week of Feb 10th

Description

In this exercise, we will explore the fundamentals of ROS (Robot Operating System) and dive into the core principles of robotic kinematics and odometry. Through hands-on tasks, you'll learn how robotic software components interact and understand how robots move and track their position.

Now, let's swim alongside ducks and start speaking their language through ROS!  

For this exercise, you'll work in a group of two, as decided in the previous exercise. We recommend you to start this assignment early as it will take some time to get used to ROS.

***TIP:** Take your time to carefully read and understand each step before executing any commands in the terminal. Pay attention to what is happening in the background of ROS—it will save you from confusion and make the exercise smoother to complete. Rushing through without understanding may lead to unnecessary complications.*

Objectives

1. Develop a strong understanding of the core components of ROS.
2. Learn the basics of mobile robot kinematics.
3. Understand mobile robot odometry.
4. Get comfortable using common and essential ROS commands.
5. Write your own code to drive a Duckiebot.
6. Gain hands-on experience with ROS nodes, services, and topics.

Hints

Familiarizing yourself with these commands, will be of great help for this exercise:

```
rostopic/rosnode/rosmmsg/rosparam/rosservice list/info
```

Prerequisites

Make sure your robot is already updated using the command:

```
dts duckiebot update csc229XX
```

Part I: Getting comfortable with ROS

In this section, we will explore the basics of ROS and prepare for hands-on implementation.

1. ROS wiki documentation:

Before starting, read the [ROS](#) wiki pages on the following topics and understand the basic concepts:

- Node - What is a ROS node, and what is its role in the ROS ecosystem?
- Topic - What are ROS topics, and how do they facilitate communication between nodes?
- Service - What are ROS services, and how do they differ from topics?
- Message - What are ROS messages, and how are they structured for communication?
- Bag - What is a ROS bag, and how is it used in data recording and playback?
- Can you describe the typical workflow for setting up communication between multiple nodes in ROS?

Answer the above questions in your report/on your webpage. Focus on understanding these basic concepts thoroughly, as they will serve as the foundation for upcoming implementation tasks.

2. Using ROS with Duckiebots:

Note: Duckiebots use ROS1, and ROS is already installed on them through the Docker files you've built earlier. You don't need to install anything extra. Let's start working with ROS!

- a. [Setup your DTROS](#) - an interface for using ROS with duckiebots
- b. Create a [catkin package](#)
- c. Write your first [publisher](#) and [subscriber](#) to communicate with each other. Add a screenshot in your report.

Side quest: After completing this, you can try the bonus question from Exercise 1 if you'd like [*not for marks*]

3. Basic operations:

- a. Store your Robot Hostname:

Write code to store your robot's hostname in a variable. Use this variable whenever you publish or subscribe to topics, instead of hardcoding the hostname. This ensures your code can run on any Duckiebot without modification. [use the same code from exercise - 1]

- b. Create your own [subscriber to the camera](#)
 - c. Process the Camera Image:
 - i. Subscribe to the camera topic you implemented in the previous step
 - ii. Print the size of each image received (height and width)
 - iii. Convert the image to grayscale using a basic OpenCV operation
 - iv. Annotate the grayscale image with the following text, where `name` with your robot's hostname and `size` is the size of the image (e.g., `640X480`)

```
Duck {name} says, 'Cheese! Capturing {size} - quack-tastic!'
```
 - v. Publish this new annotated image on a custom topic.
 - d. Use `rqt_image_view` to view the camera image on your custom topic. Take a screenshot of the viewer and include it in your report.
-

Part II: Odometry using wheel encoders

Now it's time to control your Duckiebot using your own code instead of keyboard controls! Before diving in, let's first get familiar with odometry using wheel encoders. Use this [resource](#) to understand the basics of the wheel odometry model for differential drive robots.

1. Learn how to [subscribe to the data from wheel encoders](#).
2. Also, learn how to [publish the data to the wheels](#).

Note: Do **NOT** directly publish the velocity and omega to the chassis using [tutorial](#) for next task

3. Straight Line Task:
 - a. Drive your Duckiebot forward for **1.25 meters** and then backward for **1.25 meters**.
 - b. Answer the following questions in your report/on your webpage:
 - i. Why is there a difference between the actual and desired location?
 - ii. What speed did you use?
 - iii. What happens when you increase or decrease the speed?

4. Rotation Task:
 - a. Make your Duckiebot rotate **90 degrees** ($\pi/2$ radians) clockwise on the spot, then rotate it back to **0 degrees** (0 radians) counterclockwise.
 - b. Answer the following in your report/on your webpage:
 - i. Did you observe any deviations in the rotation?
 - ii. If deviations exist, what could be the cause?
5. Does your program exit properly after you finish all the above tasks? Is it still running even if you shut down the program? If this is the case, think about how to resolve this issue.
6. [Bag](#) is file format in ROS for storing ROS messages.
 - a. Save the odometry information (the messages you're publishing) to a rosbag file
 - b. Load and Plot Trajectory
 - i. Load the rosbag file in a Python script [you can refer [this](#) or [this](#)]
 - ii. Plot the trajectory (x, y) that your Duckiebot followed

Hint: The messages you receive will include velocity(v) and angular velocity (ω). Use the kinematic equations to calculate (x, y, θ) from the velocity and omega values.

Part III: Playing with Duckiebots

In this task, you will use wheel encoders for odometry to make your Duckiebot follow a **"D"-shaped path**, D for Duckiebot. Along the way, you'll implement ROS services and LED light indications to represent different states. You'll also save your odometry data to a ROS bag and analyze it later.

General Requirements:

1. Implement a ROS service to change the LED light color of your Duckiebot for visual indication of its current state.
 - a. For example, you can use blue for one state and green for another
 - b. Choose any colors you like, as long as they are distinct between states
 - c. Clearly document the mapping of states to LED colors in your report
2. Your implementation must include at least two ROS nodes:
 - a. A **main node** to execute the task logic
 - b. A **service node** to control the LED lights
3. Ensure that all nodes terminate properly after completing the task.
4. Initial Setup: Start your Duckiebot at a designated position (provided by TAs)

5. **State 1:** Stop
 - a. Keep the robot stationary for **5 seconds**.
 - b. Set the LED lights to your chosen color for this state.
 6. **State 2:** Tracing "**D**" path
 - a. Switch the LED light to some other color
 - b. Follow a D-shaped path using wheel encoder odometry
 - i. **Straight Segment:** Move straight forward for **1 meters**
 - ii. **Semi-Circular Segment:** Perform a clockwise semi-circle to form the curved part of the D
 - iii. Keep the LED lights consistent for the entire state
 7. **State 3:** Return to the starting position
 - a. Move the robot back to the starting position and orientation
 - b. Wait for **5 seconds** at the starting position and set the LED lights to the State1 color
 8. Save the entire odometry data (wheel encoder readings) to a ROS bag file and shut down all the nodes.
 9. Use a Python script to load the ROS bag and plot the trajectory of your robot, showing the D-shaped path. Just use the same code from the previous part.
 10. Print the total execution time for the task and add it in the report.
 11. Is your duckiebot able to track what you planned according to your plan? If not, then why?
-

Bonus Questions:

1. **Reverse Parking:**
 - a. Can you drive your vehicle in reverse direction? Move straight for 0.5m, rotate anti-clockwise 90 degrees and drive in reverse direction for 0.3m. This is more like a *reverse-parking* scenario.
 - b. Modify the task to trace a different shape (e.g., an "8" or a square). What changes are needed in your code? How does the new shape affect odometry accuracy?
 2. **Frames:**
 - a. Can you make the odometry better using robot frames transformation? You can refer to this transformation link to understand how [transformations](#) work. Can you connect your robot and world frame and finish all the objectives using TF.
-

Code Guidelines

We are providing you with a template code and directory structure to help you get started quickly and easily. The zip file for the template code is on [#exercise2](#) channel on [CMPUT 412/503 W25](#) slack. The provided function blocks are meant to be flexible guidelines that you can modify, replace, or expand as needed. Think of them as a starting point that saves you time and provides a basic framework, but not as rigid rules. You're free to adapt the code to best suit your project's specific requirements, making development more straightforward and allowing you to focus on solving your unique challenges. These codeblocks can be directly used in the upcoming assignments as well.

Deliverables

Include in your written report:

- A screenshot of your subscriber info from "Hello from..."
- A screenshot of your robot's camera image view in your annotated and customized camera image
- A video of your duckiebot driving forwards then backwards for 1.25 meters
- A video of your duckiebot rotating 90 degrees and back
- A video of your duckiebot driving in a "D" shape
- LED mapping criteria
- Your rosbag plot
- Your answers to the questions included in the procedures above
- Each of the above should include a short write up explaining the video or image.
- A short overall write up about what you implemented, how it works, what you learned, what challenges you came across, and how you overcame the challenges
- Your reflections, thoughts, and comments on this exercise of the lab

On eClass you will submit:

- To submit your website written report, first publish it to the public then upload a pdf printout of your published report on eClass by the deadline (don't worry if the formatting is weird)
- A link to your course website with the written report for this exercise
- A link to your course Github repository with all the code from this exercise neatly in a subfolder

Resources

You can use any material on the internet as long as you cite it properly. This includes tools like chatGPT. You are encouraged to collaborate with your labmates and if you develop a solution together please acknowledge who you worked with in your written report.

Marking Guide

<u>Objective</u>		<u>Points</u>
Website		/1
Set Up Repo (Git)		/6
	Code to customize and annotate camera image	/0.5
	Code for the duckiebot to drive forwards and backwards 1.25 meters (using ros)	/1
	Code for duckiebot to rotate (using ros)	/1
	Code to make the duckiebot travel in a D shape using LEDs to signal state	/2
	Code to plot your tracked trajectory from rosbag files	/1
	All nodes shutdown properly	/0.25
	Updated Readme	/0.25
Report		/7
ROS basics		/2
	Difference between ROS topics, nodes, services, messages, and bags	/1
	Screenshot of subscriber info from "Hello from..." and caption explaining your image	/0.5
	A screenshot of your annotated custom camera image and caption explaining your image	/0.5
Move the Duckiebot		/1.5

	Video of duckiebot driving forward then backwards and caption explaining your video	/0.5
	Why is there a difference between the actual and desired location?	/0.5
	What speed did you use?	/0.25
	What happens when you increase or decrease the speed?	/0.25
Turn the Duckiebot		/1
	Video of duckiebot rotating clockwise, then back and caption explaining your video	/0.5
	What could be the cause of deviations in the rotation?	/0.5
Plotting the tracked trajectory		/1
	The figure output by the rosbag plotting code and caption explaining your image	/1
Duckiebot Pathing		/1.5
	Video of the duckiebot driving in a "D" shape and caption explaining your video	/1
	LEDs indicating state correctly and LED meaning are indicated	/0.5
Technical Marks		/4
	Write-up	/2
	Spelling and Grammar	/1
	References	/1
	Bonus: Reverse Parking (code and video)	/up to +1

	Bonus: Frames (code)	/up to +1
	What didn't work (bonus)*	/up to 0.5 per incomplete robot based objective above
Total	/18	

*If there is any objective you cannot complete, you have another way. In the "What I Learned" portion include a subsection for any objective or demonstration you cannot complete and include answers the following questions:

What was the barrier to completion?

What was the cause of that barrier?

What would you do next if you had more time?

Why is that what you would do next?

Anything else you learned while approaching this challenge.

Note that these bonus marks are unlikely to be given if the barrier is simply "I needed more time". They are intended to allow you to explore problems and errors, come to an understanding, and share that learning with us to earn marks you lost because those errors prevented you from completing an objective.